# Prozeduren und Funktionen

# Unterprogramme – Prozeduren und Funktionen (1)

- Unterprogramme definieren ***Anweisungsblöcke***, die über einen Namen angesprochen und beliebig oft ausgeführt werden können
- können ***Parameter*** haben
  - ermöglichen die Ausführung von Unterprogrammen mit verschiedenartigen Werten
- werden *einmal* ***deklariert***
  - Umsetzung des Algorithmus
  - erscheinen daher nicht x-mal vollständig im Quellcode
  - Deklarieren der lokalen Variablen und ***formalen Parameter***
- ist ein Unterprogramm aufgerufen worden (wird ***ausgeführt,*** ist aktiviert)
  - werden die ***aktuellen Parameter*** übergeben
  - dann wird der rufende Programmteil suspendiert bis die Prozedur fertig ist
  - nach Ende des Unterprogramms erfolgt Rückgabe der Kontrolle an den rufenden Programmteil

# Unterprogramme – Prozeduren und Funktionen (2)

- Unterscheidung in
  - ***Prozeduren***
    - überall dort aufrufbar, wo eine Anweisung stehen kann
  - ***Funktionen***
    - liefert einen Wert (mit return in C)
    - meist nur durch den Ergebnistyp von der Prozedur syntaktisch unterschieden
    - kann als Anweisung oder als Teil eines Ausdrucks aufgerufen werden
- Unterprogramme haben wie Blöcke üblicherweise ***kein Gedächtnis***
  - lokale Variablen werden mit dem Ende der Ausführung vernichtet
  - Parameter agieren bzgl. ihrer Lebensdauer wie lokale Variablen
- Prozeduren sind in C als Funktionen ohne Rückgabewert gegeben

# Parameterlose Prozeduren

## Beispiel: Ausgabe einer Überschrift

```
/* Datei: test.c */
#include <stdio.h>

        void printHeader(void) {
             printf("Artikelliste \n");
             printf("------------ \n");
        }

        int main (void) {
             printHeader();
             ...
             printHeader();
             …
              return 0;
        }
```

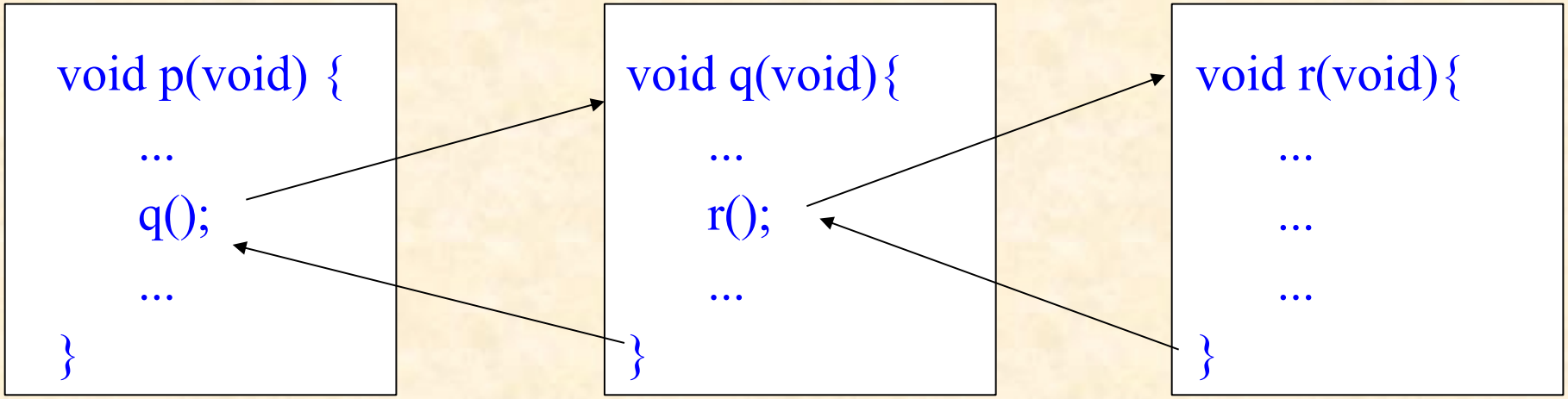
Prozedurkopf

Prozedurrumpf

Prozeduraufruf

Prozeduren werden in C als Funktionen mit speziellem Rückgabetyp beschrieben

# Wie funktioniert ein Prozeduraufruf?




## Namenskonventionen für Prozeduren

Namen sollen mit Verb und Kleinbuchstaben beginnen

Beispiele: printHeader, findMaximum, traverseList, ...

# Parameter

## Werte, die vom Aufrufer an die Methode übergeben werden

```
void printMax (int x, int y) {
    if (x > y) printf("%d",x); else printf("%d",y);
}

int main (void) {
    ...
    printMax(100, 2 * i);
    return 0;
}
```

***formale Parameter***
- im Prozedurkopf (hier x,y)
- sind Variablen der Funktion

***aktuelle Parameter***
- in Aufrufstelle (hier 100,2*i)
- können bel. Ausdrücke sein

## Parameterübergabe in C

Aktuelle Parameter werden formalen Parametern zugewiesen (x = 100; y = 2 * i;)

⇒ aktuelle Parameter müssen mit formalen verträglich sein

⇒ werden wenn nötig implizit in formale Parametertypen überführt

⇒ formale Parameter enthalten Kopien der aktuellen Parameter

# Mechanismen zur Parameterübergabe (allgemein)

- **Wertparameter** (call-by-value)
  - eine Kopie des Werts des aktuellen Parameters wird übergeben
  - der aktuelle Parameter bleibt von Veränderungen im Unterprogramm unberührt
- **Variablen- oder Referenzparameter** (call-by-reference)
  - eine Referenz auf den aktuellen Parameter wird übergeben
  - der aktuelle Parameter muß eine schreibbare Variable des gleichen Typs sein
  - der aktuelle Parameter wird durch Änderungen im Unterprogramm betroffen
- **Namensparameter** (call-by-name)
  - nicht ausgewerteter Ausdruck wird als Parameter übergeben (u.a. in Algol 60)
  - beim Zugriff auf den formalen Parameter, wird der als aktueller Parameter übergebene Ausdruck ausgewertet (***spätes Binden***)
  - Art textueller Substitution von formalen Parametern
  - Bekannt als „lazy evaluation“ in funktionalen Sprachen
- **In-, Out- und In-Out-Parameter**
  - Referenzparameter werden nach Zweck unterschieden, z.B. in verwendet in Ada

**C verwendet ausschließlich call-by-value! Aber:**
**call-by-reference sowie in/out/inout lassen sich einfach abbilden.**

# Zeigertypen als formale Parametertypen

- call-by-reference-Verhalten kann in C erreicht werden, durch Verwendung von Zeigertypen in formalen Parameterdefinitionen

```
/* Datei: ref.c */
#include <stdio.h>

    void init(int * alpha){
      *alpha = 10;
    }

    int main (void){
       int a;
       init(&a);
       printf("Der Wert von a ist %d. \n", a);
       return 0;
    }
```

Ausgabe: Der Wert von a ist 10.

# Zeigertypen als formale Parametertypen

- In/out/inout Intentionen können als Kommentare hinzugefügt werden, sind aber dann natürlich nicht bindend:

```
#include <stdio.h>
   void foo( /* IN */ int *alpha,
             /* OUT */ int *beta,
             /* INOUT */ int *gamma) {
     printf("Alpha: %d. \n", *alpha);
     *beta = 10;
      *gamma = *gamma + 1;
   }
   int main (void){
     int a=1, b, c=2;
     foo( &a, &b, &c);
     return 0;
   }
```

# Const-Parameter in C

- Formalen parametern können in C neue Werte zugewiesen werden. Ist das nicht gewünscht, hilft das Schlüsselwort const:

```
/* Datei: sum.c */
#include <stdio.h>

    void sum(const int n){
      int i = 1;
      int sum = 0;
      n = 0;    // Zuweisung führt zur Fehlermeldung

       while(i<= n){
          sum = sum + i;
          i++;
      }
      printf("%d", sum);
    }
```

Achtung: const-safe-Programmierung kann Fehleranfälligkeit der Programme mindern

# Const-Safe-Programmierung mit Zeigertypen

- const kann auch bei der Verwendung von Zeigertypen in formalen Parameterdefinitionen benutzt werden

```
/* Datei: ref.c */
#include <stdio.h>

    void init(const int *alpha){ // Zeiger auf konstantem int-Wert
      *alpha = 10; // Fehlermeldung
    }

    int main (void){
       int a;
       init(&a);
       printf("Der Wert von a ist %d. \n", a);
       return 0;
    }
```

# Const-Safe-Programmierung mit Zeigertypen

- const kann auch bei der Verwendung von Zeigertypen in formalen Parameterdefinitionen benutzt werden

```
/* Datei: ref.c */
#include <stdio.h>

   void init(int * const alpha){ // nun ist Zeiger als konstant definiert,
                                 // nicht aber der int-Wert
     *alpha = 10; // keine Fehlermeldung
   }

   int main (void){
      int a;
      init(&a);
      printf("Der Wert von a ist %d. \n", a);
      return 0;
   }
```

Ausgabe: Der Wert von a ist 10.

# Funktionen

Unterprogramme, die einen Ergebniswert an den Aufrufer zurückliefern

```
#include <stdio.h>
int max (int x, int y) {
    if (x > y) return x; else return y;
}
int main (void) {
    ...
    printf("%d",  max(100, i + j) + 1);
    return 0;
}
```

- haben **Ergebnistyp** (hier int) statt void (= kein Typ)
- liefern Ergebnis mittels return-Anweisung an den Aufrufer
- werden wie Operanden in einem Ausdruck benutzt

# Lokale und globale Variablendeklarationen

## Funktionen können lokale Variablendeklarationen enthalten

```
int log2 (int x) {
        int res = 0;
        while (x > 1) {
                x = x / 2;
                res++;
        }
        return res;
}
int y; //oder auch extern int y;
int main (void) {
        int n;
        scanf("%d", &n);
        int x = log2(n);
        printf("%d", x);
        return 0;
}
```

### Lokale Variablen

- sollten vor Gebrauch initialisiert werden
- können ***nur*** innerhalb der Prozedur/Funktion verwendet werden
- werden **bei jedem Aufruf neu angelegt** und **am Ende der Funktion wieder freigegeben**

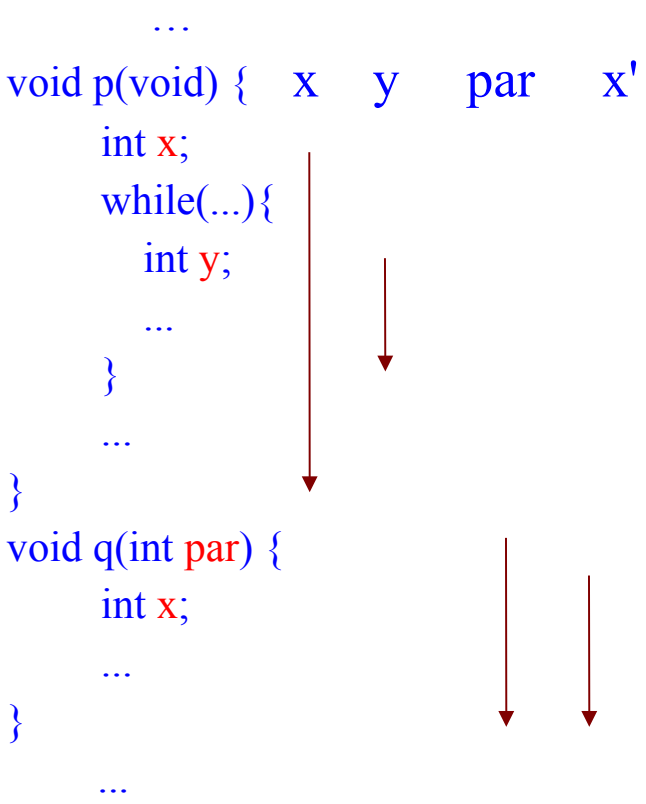
### Globale (externe) Variablen

- werden automatisch initialisiert
- werden in Dateien außerhalb von Prozeduren/ Funktionen definiert
- können nach Definition in nachfolgenden Prozeduren/Funktionen verwendet werden
- werden **beim Programmstart angelegt** und erst **am Ende des Programms freigegeben**
- softwaretechnisch jedoch nie eine gute Lösung, sollten daher vermieden werden

# Sichtbarkeitsbereich von Variablen

Programmstück, in dem auf Variablen zugegriffen werden kann

(auch Gültigkeitsbereich oder Scope genannt)



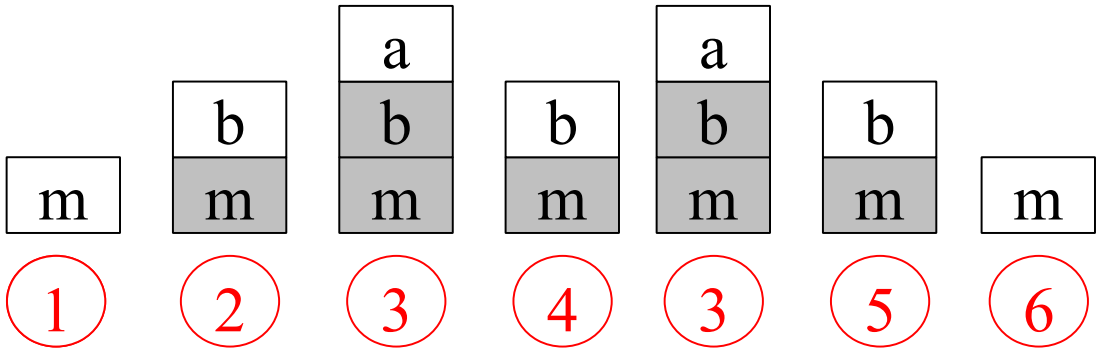

## Regeln in C:

1. Variablen dürfen in einem Block nicht mehrmals deklariert werden (jedoch ist ein Überdecken in inneren Blöcken erlaubt).
2. Sichtbarkeitsbereich lokaler Variablen beginnt mit ihrer Deklaration und geht bis ans Ende des Blocks in dem sie definiert sind.
3. Sichtbarkeitsbereich globaler Variablen beginnt mit der Deklaration und geht bis ans Ende der Datei, in der sie definiert sind.

# Lebensdauer lokaler Variablen

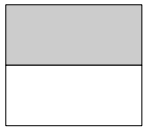
Lokale Variablen oder Variablen innerer Blöcke werden angelegt, wenn ihr Block betreten wird und freigegeben, wenn ihr Block verlassen wird.

```
/* Datei: LifenessDemo.c */
      void A(void) {
            int a;
      3     ...
      }
      void B(void) {
            int b;
      2     ... A();  4  ... A();     5        ...
      }
      int main(void) {
            int m;
      1     ... B(); ...   6
      }
}
```

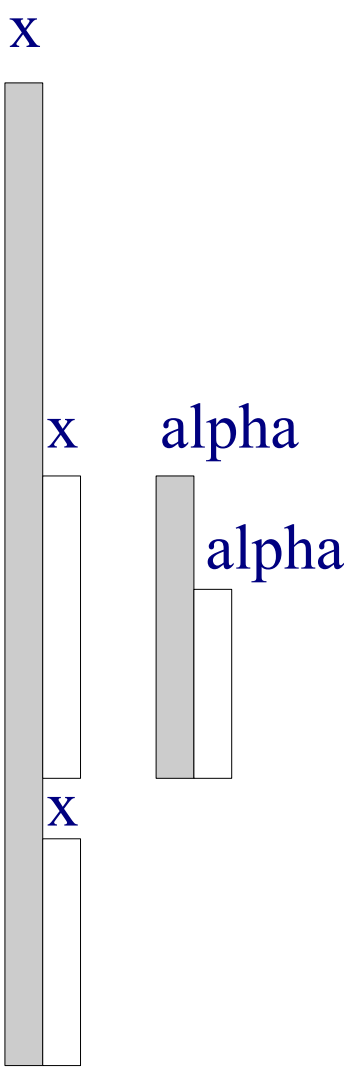


# Lebensdauer und Sichtbarkeit: globale Variablen

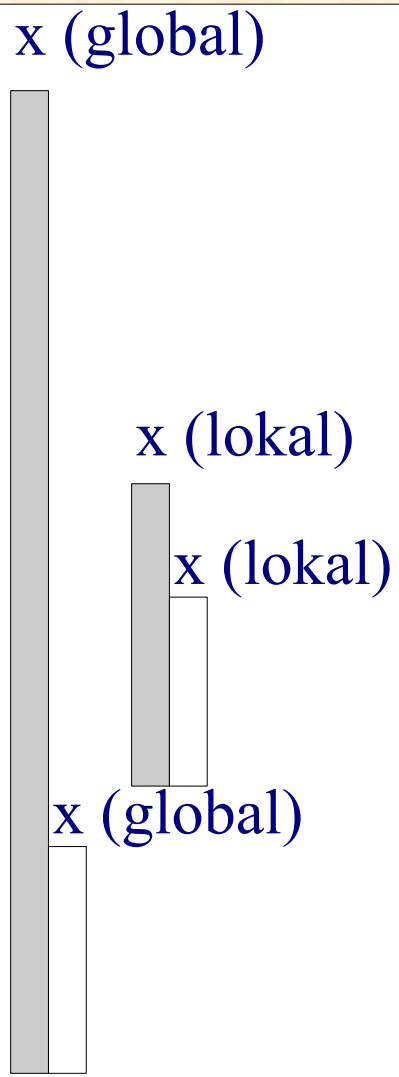
```
#include <stdio.h>
int main(void) {
      ...
}
int x;
void sub1(void) {
       int alpha = 4;
            ...
}
void sub2(void) {
      ...
}
```



Lebensdauer
Sichtbarkeit




```
#include <stdio.h>
int main(void) {
      ...
}
int x;
void sub1(void) {
       int x;
            ...
}
void sub2(void) {
      ...
}
```



# Vorwärtsdeklaration von Funktionen

- Vorwärtsdeklarationen werden verwendet um Funktionen bereits vor deren Deklaration aufrufen zu können
- Funktionsprototyp kann abweichend zur Funktionsdeklaration sein

  - formale Parameternamen müssen nicht identisch sein bzw. können sogar weggelassen werden

```
#include <stdio.h>
void init(int * beta);   // hier auch korrekt: void init(int *);
int main (void){
  int a;
  init(&a);
  printf("Der Wert von a ist %d. \n", a);
  return 0;
}
void init(int * alpha){
  *alpha = 10;
}
```

Stimmen nachfolgende Aufrufe bzw. eigentliche Deklaration von init nicht mit Prototypen überein, gibt der Übersetzer eine Fehlermeldung aus.

# Return in Prozeduren

```
#include <stdio.h>
  void printLog2 (int x) {
     if (x <= 0) return; // kehrt zum Rufer zurück
     int res = 0;
     while (x > 1) {x = x / 2; res++;}
     printf("%d \n", res);
  }
  int main (void) {
          int x;
          scanf(“%d “, &x);
          printLog2(x);
          return 0;
  }
```

Funktionen ***müssen*** mit return beendet werden

Prozeduren ***können*** mit return beendet werden

# Beispiele

# Beispiele: Prozeduren und Funktionen

Größter gemeinsamer Teiler nach Euklid

```
int ggt (int x, int y) {
    int rest = x % y;
    while (rest != 0) {
        x = y; y = rest; rest = x % y;
    }
    return y;
}
```

Kürzen eines Bruchs

```
void reduce (int z, int n) {
    int x = ggt(z, n);
    printf("%d", z/x);
    printf("/");
    printf("%d", n/x);
}
```

# Beispiele: Prozeduren und Funktionen

Prüfe, ob eine Zahl x eine Primzahl ist

```
_Bool isPrime (int x) {
    if (x == 1) return false;
    if (x == 2) return true;
    if (x % 2 == 0)
        return false;
    int i = 3;

    while (i * i <= x) {
        if (x % i == 0)
            return false;
        i = i + 2;
    }

    // i > √x und x ist durch keine Zahl
    // kleiner als i und größer als 1 teilbar

    return true;
}
```

#include <stdbool.h> notwendig

# Zeiger auf Funktionen

# Zeiger auf Funktionen

- Zeigertyp, der in einer Variablen die Adresse einer Funktion speichert
  - hiermit können Funktionen als Parameter an Funktionen übergeben werden
- Vereinbarung eines Zeigers auf Funktionen

  Definition:

```
int (*ptr) (char); // Zeiger auf eine Funktion: int funktionsname (char){...}
```

  Zuweisung einer Funktionsadresse:

```
ptr = &funktionsname; /* oder */ ptr = funktionsname;
```

- Aufruf der Funktion über Dereferenzierungsoperator oder Zeigernamen

```
…
int main(void){
    …
    int a;
    a = funktionsname('A');
    …
}
```

```
...
int main(void){
    ptr = funktionsname;
    ...
    int a;
    a = (*ptr) ('A');
    // hier auch möglich: a = ptr('A'); ...}
```

# Beispiel: Zeiger auf Funktionen

```
void f1(void){
        printf("Druecken Sie bitte Enter:");
        getchar();
}
double eval_time(void (*ptr)(void)){
        // double eval_time(void ptr(void)) ist äquivalent
        time_t begin, end;
        begin = time(NULL);
        (*ptr) ();
        end = time(NULL);
        return difftime(end, begin);
  }
int main(void){
        printf("Zeit bis Enter gedrueckt wurde: %3.0f sec\n", eval_time(f1));
        printf("Zeit bis Enter gedrueckt wurde: %3.0f sec\n", eval_time(f1));
        return 0;
}
```

eval_time() misst Durchlaufzeit der übergebenen Funktion

# Programme aus mehreren Dateien

# Programme aus mehreren Dateien – extern-Deklarationen

Zugriff auf Funktionen und globalen Variablen aus anderen Dateien unter Verwendung von extern-Deklarationen

```
/* Datei: ext1.c */
#include <stdio.h>
extern void f2(void);
void f1(void);
extern int zahl;
int main(void){
        printf("\n hier ist main, zahl = %d", zahl);
        f1();
        f2();
        return 0;
}
int zahl = 6;
void f1(void){
      printf("\n hier ist f1, zahl = %d", zahl);
}
```

# Programme aus mehreren Dateien – extern-Deklarationen

```
/* Datei: ext2.c */
#include <stdio.h>
extern int zahl;
void f2(void){
        printf("\n hier ist f2, zahl = %d", zahl);
}
```

Die Programmausgabe ist:

```
hier ist main, zahl = 6
hier ist f1, zahl = 6
hier ist f2, zahl = 6
```

Merke: externe Funktionsdeklarationen können ohne ***extern*** definiert sein, da Funktionsprototypen automatisch als extern definiert werden

# Verwendung von Header-Dateien

- Extern-Deklarationen der Bibliotheksfunktionen sind in Header-Dateien definiert, die mit einem Präprozessorbefehl eingebunden werden können

```
#include <stdio.h>
```

- um Anwendern eigene Funktionen einbinden zu lassen, bietet sich auch die Definition von Header-Dateien an
  - damit ist gewährleistet, dass nur eine Extern-Deklaration existiert, die überall genutzt werden kann, wo die Funktion aufgerufen wird
  - folgende Regeln ermöglichen eine größtmögliche Konsistenz über Dateigrenzen hinweg
    - jede Funktion, die außerhalb benutzt werden soll, erhält eine Extern-Deklaration in einer Header-Datei
    - jede Datei, die einen Aufruf einer solchen Funktion enthält, inkludiert die entsprechende Header-Datei
    - die Datei, die die Definition der Funktionen enthält, soll die Header-Datei ebenfalls inkludieren

# Beispiel: Verwendung eigener Header-Dateien

```
/* Datei: ext.h */
extern void f2(void);
extern void f1(void);
extern int zahl;
/* Datei: ext1.c */
#include <stdio.h>
#include "ext.h"
int main(void){
        printf("\n hier ist main, zahl = %d", zahl);
        f1();
        f2();
        return 0;
}
int zahl = 6;
void f1(){
        printf("\n hier ist f1, zahl = %d", zahl);
}
```

# Beispiel: Verwendung eigener Header-Dateien

```
/* Datei: ext2.c */
#include <stdio.h>
#include "ext.h"
void f2(void){
        printf("\n hier ist f2, zahl = %d", zahl);
}
```

Die Programmausgabe ist:

```
hier ist main, zahl = 6
hier ist f1, zahl = 6
hier ist f2, zahl = 6
```